Project overview

Download Ontario Hansard HTML pages + parse them into structured “rows” in Mongo.

Pipeline Overview:

1. the downloader discovers session/day pages and stores raw HTML snapshots in MongoDB
2. the incremental downloader checks for newly added day pages and stores only the new ones
3. the parser reads from hansard-html-snapshots, selects only documents that have not yet been parsed, extracts structured rows, and writes them to hansard-parsed-rows
4. After parsing, the corresponding HTML snapshot document is updated with parsing metadata i.e. if there were any errors in parsing or not
5. 32nd, 37th, 38th and 39th parliament have a different html format and are treated as edge cases here.
6. (future) joins to “offices” dataset using speaker name matching

Data storage in MongoDB:

Database: `case-scraping`

Collections created/used:

- **Hansard-html-snapshots:** Stores the fetched day HTML pages.
    - key fields: batch, url
    - stored fields: final_url, status, headers, fetched_at, content (bytes), pdf_links, parsed, parsed_error, parsed_at
- **Hansard-pdf-metadata**: Stores PDF link metadata (not the PDF bytes).
    - key fields: batch, pdf_url
    - stored fields: day_url, final_url, status, headers, fetched_at
- **Hansard-parsed-rows:** Stores parsed transcript “rows”
    - unique index: (source_url, ID)
    - stored fields: ID, Date, OrderofBusiness, SubjectofBusiness, PersonSpeaking, Intervention, source_url,

Downloader

Input: Root URL: https://www.ola.org/en/legislative-business/house-documents

Discovery logic

- discover_session_urls() uses SESSION_RE to find session links
- discover_hansard_day_urls() uses HANSARD_DAY_RE to find day Hansard pages inside each session
- Dedupes day URLs before downloading

Fetching

- fetch() retries up to 4 times
- Retries on: 429 / 502 / 503 / 504
- Adds sleep/jitter between requests and on retries (sleep())

What gets written to Mongo

- save_day_html_snapshot() writes raw HTML bytes into content
- It also stores pdf_links found on that day page if present.
- save_pdf_metadata() writes PDF metadata.

Logging

- LOGS/downloader.log
- document what “success/failure” looks like and what gets logged

Parser

Input: Mongo collection: hansard-html-snapshots

Output row schema:

Each parsed row is:

- ID (date + sequence): YYYYMMDD-0000001
- Date (YYYY-MM-DD)
- OrderofBusiness (from <h2>)
- SubjectofBusiness (from <h3>/<h4> and some heading heuristics)

- PersonSpeaking (speaker label or "Procedure")
- Intervention (merged text for that segment)
- source_url (original day page URL)

How parsing works

1) Find transcript container
    a) Prefer , fallback to <article>
    b) Remove TOC (div#toc) if present

2) Date extraction
    a) from <time class="datetime" datetime="...">
    b) fallback from canonical URL

3) Track hierarchy
    a) <h2> sets current_order
    b) <h3> / <h4> sets current_subject
    c) In hansards from 32nd, 37th, 38th and 39th parliament header tags dont exist, but subjects were specified using all caps lettering. so the function is_caps_heading_paragraph() extracts the current subject

4) Paragraph handling
    a) skip empty lines
    b) skip “time-only” paragraphs like 1030

5) Speaker detection (two patterns)
    a) If <p class="speakerStart">: parse_speaker() extracts label using regex that captures text before colon
    b) If no speakerStart, fallback: extract_strong_speaker_label() detects Some Name:

6) Speaker turn boundaries
    a) When new speaker stars, flush(), update current_person
    b) Continuation paragraphs (plain <p>) are assumed to belong to the current speaker unless detected as procedural

7) Procedural detection
    a) looks_procedural() function uses regex list (interjections, applause, house adjourned, etc.)
    b) If a procedural line appears mid-speech: flush current speaker segment, emit that line as PersonSpeaking="Procedure", restore the speaker afterward

    c) From certain sessions onward, procedural lines are marked in the HTML using a procedural class, so the parser uses that class to identify those lines.

8) flush() behavior
    a) builds one row per accumulated buffer
    b) increments seq number
    c) assigns PersonSpeaking as current speaker, or "Procedure" if none

Edge case paragraph:

A key parsing challenge is that Hansard HTML does not consistently encode speaker-turn boundaries. While the beginning of a speaker's remarks is often marked with a speakerStart paragraph, subsequent paragraphs from the same speaker appear as plain <p> elements without metadata. Procedural statements (e.g., interjections, stage directions) are also not always explicitly tagged. To handle this, the parser assumes untagged paragraphs continue the current speaker's remarks unless the paragraph matches known procedural patterns, in which case it is emitted as "Procedure".

Edge cases:

1. 32nd, 37th, 38th and 39th parliament style transcripts
    a. no explicit speakerStart tag
    b. speaker label appears as Name: at paragraph start
    c. The fallback extractor handles this

2. No explicit "procedure" tags in older hansards
    a. parser uses phrase-based detection (PROCEDURAL_START_RE, PROCEDURAL_END_RE)

3. Speaker label appears twice for transcripts that have the speakerStart tag and the strong tag:
    a. you strip "Label:" from the text, and if still duplicates it you remove the tag and re-extract text

Incremental downloader and parser updates

To support daily automation, I created the incremental downloader in a separate file from the full downloader. The purpose of this file is to fetch only newly added Hansard HTML pages. I also

added an index for scraped URLs stored in MongoDB so the pipeline can efficiently check which URLs already exist in the database and avoid inserting duplicates in future runs.

Instead of scanning all historical sessions, the incremental downloader fetches the recent House Documents link from the root Hansard page, since the sessions from the current Parliament will appear there. This discovery step is handled in the discover_recent_recent_house_documents() function.

To support incremental parsing, I added parsing-status fields to the hansard-html-snapshots collection in MongoDB. I then updated the parser so that, before parsing begins, it checks the parsing-status field in each HTML snapshot document. Only documents that have not yet been parsed are selected for parsing.

So the workflow in this stage is:

1. run the incremental downloader to fetch only newly added HTML files
2. run the parser to parse only the HTML files whose parsed field is still false or unset

Commons file

I moved several reusable helper functions into a shared commons file so they can be imported and reused across multiple scripts in the pipeline.